Scenario 1: emergency rescue priority system

```
#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_REQUESTS 100
#define MAX_TEAMS 20
#define ID_LENGTH 20
#define TEXT_LENGTH 80
#define STATUS_LENGTH 20

typedef enum {
    LEVEL_CRITICAL = 4,
    LEVEL_HIGH = 3,
    LEVEL_MODERATE = 2,
    LEVEL_LOW = 1
} EmergencyLevel;

typedef struct {
    char id[ID_LENGTH];
    char location[TEXT_LENGTH];
    int people;
    EmergencyLevel level;
    char resources[TEXT_LENGTH];
    int team_index;
    char status[STATUS_LENGTH];
} RescueRequest;

typedef struct {
    char name[TEXT_LENGTH];
    int available;
} RescueTeam;

static RescueRequest requests[MAX_REQUESTS];
static RescueTeam teams[MAX_TEAMS];
static int request_count;
static int team_count;

static int read_line(const char *prompt, char *buffer, size_t size)
{
    size_t length;
```

```
    printf("%s", prompt);
    if (fgets(buffer, (int)size, stdin) == NULL) {
        return 0;
    }

    length = strlen(buffer);
    if (length > 0 && buffer[length - 1] == '\n') {
        buffer[length - 1] = '\0';
    } else {
        int character;
        while ((character = getchar()) != '\n' && character != EOF) {
        }
    }

    return 1;
}

static int read_integer(const char *prompt, int minimum, int maximum, int *value)
{
    char input[TEXT_LENGTH];
    char *end;
    long parsed;

    if (!read_line(prompt, input, sizeof(input))) {
        return 0;
    }

    parsed = strtol(input, &end, 10);
    while (isspace((unsigned char)*end)) {
        end++;
    }
    if (input[0] == '\0' || *end != '\0' || parsed < minimum || parsed > maximum) {
        printf("Enter a number from %d to %d.\n", minimum, maximum);
        return -1;
    }

    *value = (int)parsed;
    return 1;
}

static int find_request(const char *id)
{
    int i;
```

```
    for (i = 0; i < request_count; i++) {
        if (strcmp(requests[i].id, id) == 0) {
            return i;
        }
    }
    return -1;
}

static const char *level_name(EmergencyLevel level)
{
    switch (level) {
    case LEVEL_CRITICAL: return "Critical";
    case LEVEL_HIGH: return "High";
    case LEVEL_MODERATE: return "Moderate";
    default: return "Low";
    }
}

static int priority_score(const RescueRequest *request)
{
    /* Severity dominates; affected people break ties within a level. */
    return (int)request->level * 1000 + request->people;
}

static void add_request(void)
{
    RescueRequest request;
    int choice;
    int result;

    if (request_count >= MAX_REQUESTS) {
        printf("The request list is full.\n");
        return;
    }

    if (!read_line("Request ID: ", request.id, sizeof(request.id))) return;
    if (request.id[0] == '\0' || find_request(request.id) >= 0) {
        printf("Request ID must be non-empty and unique.\n");
        return;
    }
    if (!read_line("Location: ", request.location, sizeof(request.location)) ||
        request.location[0] == '\0') {
        printf("Location cannot be empty.\n");
```

```
        return;
    }
    result = read_integer("Number of people affected: ", 1, 1000000, &request.people);
    if (result != 1) return;

    printf("Emergency level: 1) Critical  2) High  3) Moderate  4) Low\n");
    result = read_integer("Choose level: ", 1, 4, &choice);
    if (result != 1) return;
    request.level = (EmergencyLevel)(5 - choice);

    if (!read_line("Required resources: ", request.resources, sizeof(request.resources))) return;
    if (request.resources[0] == '\0') {
        strcpy(request.resources, "Not specified");
    }

    request.team_index = -1;
    strcpy(request.status, "Pending");
    requests[request_count++] = request;
    printf("Request %s registered. Priority score: %d\n",
           request.id, priority_score(&request));
}

static void print_request(const RescueRequest *request)
{
    printf("\nRequest ID : %s\n", request->id);
    printf("Location   : %s\n", request->location);
    printf("People     : %d\n", request->people);
    printf("Emergency  : %s\n", level_name(request->level));
    printf("Priority   : %d\n", priority_score(request));
    printf("Resources  : %s\n", request->resources);
    printf("Team       : %s\n", request->team_index < 0
           ? "Unassigned" : teams[request->team_index].name);
    printf("Status     : %s\n", request->status);
}

static void display_by_priority(void)
{
    int order[MAX_REQUESTS];
    int i;
    int j;

    if (request_count == 0) {
        printf("No rescue requests have been registered.\n");
        return;
```

```
    }

    for (i = 0; i < request_count; i++) {
        order[i] = i;
    }
    for (i = 1; i < request_count; i++) {
        int current = order[i];
        j = i - 1;
        while (j >= 0 && priority_score(&requests[order[j]]) <
                         priority_score(&requests[current])) {
            order[j + 1] = order[j];
            j--;
        }
        order[j + 1] = current;
    }

    printf("\n--- Rescue requests (highest priority first) ---\n");
    for (i = 0; i < request_count; i++) {
        print_request(&requests[order[i]]);
    }
}

static void add_team(void)
{
    RescueTeam team;

    if (team_count >= MAX_TEAMS) {
        printf("The rescue team list is full.\n");
        return;
    }
    if (!read_line("Team name: ", team.name, sizeof(team.name))) return;
    if (team.name[0] == '\0') {
        printf("Team name cannot be empty.\n");
        return;
    }

    team.available = 1;
    teams[team_count++] = team;
    printf("Team %s added and marked available.\n", team.name);
}

static void assign_team(void)
{
    char id[ID_LENGTH];
```

```
int request_index;
int choice;
int available_indices[MAX_TEAMS];
int available_count = 0;
int i;
int result;

if (!read_line("Request ID to assign: ", id, sizeof(id))) return;
request_index = find_request(id);
if (request_index < 0) {
    printf("Request not found.\n");
    return;
}
if (strcmp(requests[request_index].status, "Resolved") == 0 ||
    strcmp(requests[request_index].status, "Cancelled") == 0) {
    printf("A resolved or cancelled request cannot be assigned.\n");
    return;
}
if (requests[request_index].team_index >= 0) {
    printf("This request already has team %s assigned.\n",
           teams[requests[request_index].team_index].name);
    return;
}

for (i = 0; i < team_count; i++) {
    if (teams[i].available) {
        available_indices[available_count++] = i;
    }
}
if (available_count == 0) {
    printf("No available teams. Add a team or release an assigned team first.\n");
    return;
}

printf("Available teams:\n");
for (i = 0; i < available_count; i++) {
    printf("%d) %s\n", i + 1, teams[available_indices[i]].name);
}
result = read_integer("Choose team: ", 1, available_count, &choice);
if (result != 1) return;

requests[request_index].team_index = available_indices[choice - 1];
teams[requests[request_index].team_index].available = 0;
printf("Team %s assigned to request %s.\n",
```

```
        teams[requests[request_index].team_index].name, id);
}

static void update_status(void)
{
    char id[ID_LENGTH];
    int request_index;
    int choice;
    int result;

    if (!read_line("Request ID to update: ", id, sizeof(id))) return;
    request_index = find_request(id);
    if (request_index < 0) {
        printf("Request not found.\n");
        return;
    }

    printf("Current status: %s\n", requests[request_index].status);
    printf("1) Pending  2) In Progress  3) Resolved  4) Cancelled\n");
    result = read_integer("Choose new status: ", 1, 4, &choice);
    if (result != 1) return;

    switch (choice) {
    case 1: strcpy(requests[request_index].status, "Pending"); break;
    case 2: strcpy(requests[request_index].status, "In Progress"); break;
    case 3: strcpy(requests[request_index].status, "Resolved"); break;
    case 4: strcpy(requests[request_index].status, "Cancelled"); break;
    }

    if ((choice == 3 || choice == 4) && requests[request_index].team_index >= 0) {
        teams[requests[request_index].team_index].available = 1;
        requests[request_index].team_index = -1;
        printf("Assigned team released.\n");
    }
    printf("Request %s status updated to %s.\n",
           id, requests[request_index].status);
}

static void search_request(void)
{
    char id[ID_LENGTH];
    int index;

    if (!read_line("Request ID to search: ", id, sizeof(id))) return;
```

```
    index = find_request(id);
    if (index < 0) {
        printf("Request not found.\n");
        return;
    }
    print_request(&requests[index]);
}

static void emergency_summary(void)
{
    int level_counts[5] = {0};
    int status_counts[4] = {0};
    long total_people = 0;
    int pending_or_active = 0;
    int i;

    for (i = 0; i < request_count; i++) {
        level_counts[requests[i].level]++;
        total_people += requests[i].people;
        if (strcmp(requests[i].status, "Pending") == 0) {
            status_counts[0]++;
            pending_or_active++;
        } else if (strcmp(requests[i].status, "In Progress") == 0) {
            status_counts[1]++;
            pending_or_active++;
        } else if (strcmp(requests[i].status, "Resolved") == 0) {
            status_counts[2]++;
        } else {
            status_counts[3]++;
        }
    }

    printf("\n--- Emergency Summary ---\n");
    printf("Total requests       : %d\n", request_count);
    printf("People affected      : %ld\n", total_people);
    printf("Critical / High      : %d\n",
           level_counts[LEVEL_CRITICAL] + level_counts[LEVEL_HIGH]);
    printf("Moderate / Low       : %d\n",
           level_counts[LEVEL_MODERATE] + level_counts[LEVEL_LOW]);
    printf("Pending              : %d\n", status_counts[0]);
    printf("In Progress          : %d\n", status_counts[1]);
    printf("Resolved             : %d\n", status_counts[2]);
    printf("Cancelled            : %d\n", status_counts[3]);
    printf("Unresolved requests  : %d\n", pending_or_active);
```

```
    printf("Available teams      : ");
    {
        int available = 0;
        for (i = 0; i < team_count; i++) {
            available += teams[i].available;
        }
        printf("%d of %d\n", available, team_count);
    }
}

int main(void)
{
    int choice;
    int result;

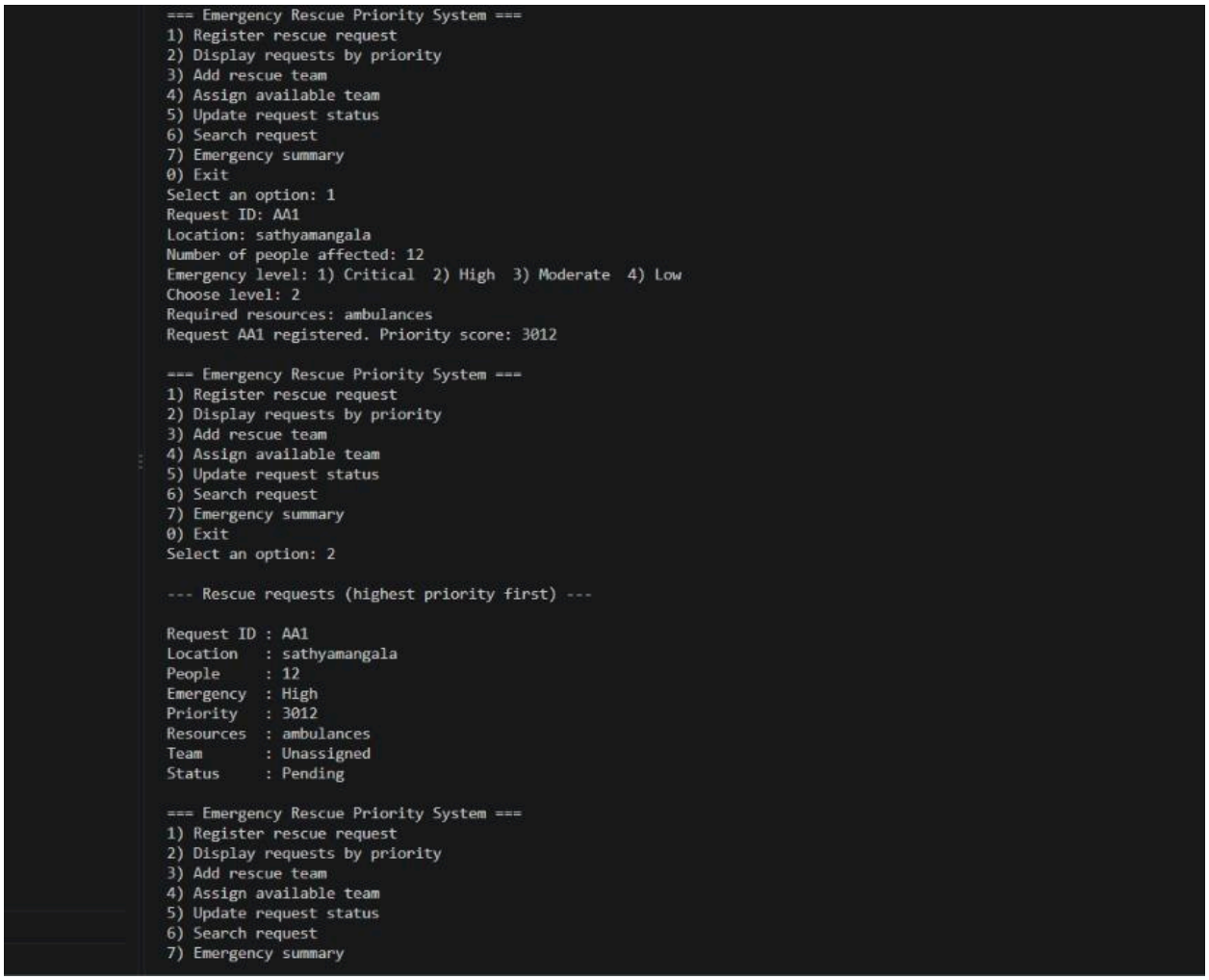
    for (;;) {
        printf("\n=== Emergency Rescue Priority System ===\n");
        printf("1) Register rescue request\n");
        printf("2) Display requests by priority\n");
        printf("3) Add rescue team\n");
        printf("4) Assign available team\n");
        printf("5) Update request status\n");
        printf("6) Search request\n");
        printf("7) Emergency summary\n");
        printf("0) Exit\n");

        result = read_integer("Select an option: ", 0, 7, &choice);
        if (result == 0) {
            printf("\nInput closed. Exiting.\n");
            break;
        }
        if (result < 0) {
            continue;
        }

        switch (choice) {
        case 1: add_request(); break;
        case 2: display_by_priority(); break;
        case 3: add_team(); break;
        case 4: assign_team(); break;
        case 5: update_status(); break;
        case 6: search_request(); break;
        case 7: emergency_summary(); break;
        case 0: return 0;
```

```
        }
    }
    return 0;
}
```

Output



Scenario 2: smart campus complaint system

```
#include <ctype.h>
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_COMPLAINTS 500
#define ID_SIZE 32
#define CATEGORY_SIZE 48
#define PROBLEM_SIZE 200
#define LOCATION_SIZE 80
#define DATA_FILE "complaints.dat"

typedef enum {
    PRIORITY_CRITICAL = 1,
    PRIORITY_HIGH,
    PRIORITY_MEDIUM,
    PRIORITY_LOW
} Priority;

typedef enum {
    STATUS_PENDING = 1,
    STATUS_IN_PROGRESS,
    STATUS_RESOLVED
} Status;

typedef struct {
    char id[ID_SIZE];
    char category[CATEGORY_SIZE];
    char problem[PROBLEM_SIZE];
    char location[LOCATION_SIZE];
    Priority priority;
    Status status;
} Complaint;

static Complaint complaints[MAX_COMPLAINTS];
static int complaintCount = 0;

static int readLine(const char *prompt, char *buffer, size_t size) {
    size_t length;
```

```
    printf("%s", prompt);
    if (fgets(buffer, (int)size, stdin) == NULL) {
        return 0;
    }

    length = strcspn(buffer, "\r\n");
    if (buffer[length] == '\0' && length == size - 1) {
        int character;
        while ((character = getchar()) != '\n' && character != EOF) {
        }
        printf("Input is too long. Please try again.\n");
        return -1;
    }

    buffer[length] = '\0';
    return 1;
}

static int readNonEmpty(const char *prompt, char *buffer, size_t size) {
    int result;
    do {
        result = readLine(prompt, buffer, size);
        if (result <= 0) {
            return result;
        }
        if (buffer[0] == '\0') {
            printf("This field cannot be empty.\n");
        }
    } while (buffer[0] == '\0');
    return 1;
}

static int readInteger(const char *prompt, int minimum, int maximum, int *value) {
    char buffer[64];
    char *end;
    long parsed;
    int result;

    for (;;) {
        result = readLine(prompt, buffer, sizeof(buffer));
        if (result == 0) {
            return 0;
        }
```

```
        if (result < 0) {
            continue;
        }

        errno = 0;
        parsed = strtol(buffer, &end, 10);
        while (isspace((unsigned char)*end)) {
            end++;
        }
        if (buffer[0] == '\0' || *end != '\0' || errno == ERANGE ||
            parsed < minimum || parsed > maximum) {
            printf("Enter a number from %d to %d.\n", minimum, maximum);
            continue;
        }
        *value = (int)parsed;
        return 1;
    }
}

static int equalsIgnoreCase(const char *left, const char *right) {
    while (*left != '\0' && *right != '\0') {
        if (tolower((unsigned char)*left) != tolower((unsigned char)*right)) {
            return 0;
        }
        left++;
        right++;
    }
    return *left == '\0' && *right == '\0';
}

static int containsIgnoreCase(const char *text, const char *query) {
    size_t queryLength = strlen(query);
    const char *start;

    if (queryLength == 0) {
        return 1;
    }
    for (start = text; *start != '\0'; start++) {
        size_t i = 0;
        while (i < queryLength && start[i] != '\0' &&
               tolower((unsigned char)start[i]) ==
                   tolower((unsigned char)query[i])) {
            i++;
        }
```

```
        if (i == queryLength) {
            return 1;
        }
    }
    return 0;
}

static const char *priorityName(Priority priority) {
    switch (priority) {
        case PRIORITY_CRITICAL: return "Critical";
        case PRIORITY_HIGH: return "High";
        case PRIORITY_MEDIUM: return "Medium";
        case PRIORITY_LOW: return "Low";
        default: return "Unknown";
    }
}

static const char *statusName(Status status) {
    switch (status) {
        case STATUS_PENDING: return "Pending";
        case STATUS_IN_PROGRESS: return "In Progress";
        case STATUS_RESOLVED: return "Resolved";
        default: return "Unknown";
    }
}

static int validComplaint(const Complaint *complaint) {
    return complaint->id[0] != '\0' &&
           complaint->category[0] != '\0' &&
           complaint->problem[0] != '\0' &&
           complaint->location[0] != '\0' &&
           complaint->priority >= PRIORITY_CRITICAL &&
           complaint->priority <= PRIORITY_LOW &&
           complaint->status >= STATUS_PENDING &&
           complaint->status <= STATUS_RESOLVED;
}

static int loadComplaints(void) {
    FILE *file = fopen(DATA_FILE, "r");
    int count;
    int i;

    if (file == NULL) {
        if (errno == ENOENT) {
```

```
        return 1;
    }
    fprintf(stderr, "Could not open %s: %s\n", DATA_FILE, strerror(errno));
    return 0;
}

if (fscanf(file, "%d\n", &count) != 1 || count < 0 ||
    count > MAX_COMPLAINTS) {
    fprintf(stderr, "The complaint data file has an invalid record count.\n");
    fclose(file);
    return 0;
}

for (i = 0; i < count; i++) {
    Complaint complaint;
    char line[PROBLEM_SIZE];
    int priority;
    int status;

    if (fgets(complaint.id, sizeof(complaint.id), file) == NULL ||
        fgets(complaint.category, sizeof(complaint.category), file) == NULL ||
        fgets(complaint.problem, sizeof(complaint.problem), file) == NULL ||
        fgets(complaint.location, sizeof(complaint.location), file) == NULL ||
        fgets(line, sizeof(line), file) == NULL ||
        sscanf(line, "%d", &priority) != 1 ||
        fgets(line, sizeof(line), file) == NULL ||
        sscanf(line, "%d", &status) != 1) {
        fprintf(stderr, "The complaint data file is incomplete or corrupted.\n");
        fclose(file);
        return 0;
    }

    complaint.id[strcspn(complaint.id, "\r\n")] = '\0';
    complaint.category[strcspn(complaint.category, "\r\n")] = '\0';
    complaint.problem[strcspn(complaint.problem, "\r\n")] = '\0';
    complaint.location[strcspn(complaint.location, "\r\n")] = '\0';
    complaint.priority = (Priority)priority;
    complaint.status = (Status)status;
    if (!validComplaint(&complaint)) {
        fprintf(stderr, "The complaint data file contains an invalid record.\n");
        fclose(file);
        return 0;
    }
    complaints[i] = complaint;
```

```
    }

    if (fclose(file) != 0) {
        fprintf(stderr, "Could not close %s after reading.\n", DATA_FILE);
        return 0;
    }
    complaintCount = count;
    return 1;
}

static int saveComplaints(void) {
    FILE *file = fopen(DATA_FILE, "w");
    int i;

    if (file == NULL) {
        fprintf(stderr, "Could not save %s: %s\n", DATA_FILE, strerror(errno));
        return 0;
    }

    if (fprintf(file, "%d\n", complaintCount) < 0) {
        fprintf(stderr, "Could not write complaint data to %s.\n", DATA_FILE);
        fclose(file);
        return 0;
    }

    for (i = 0; i < complaintCount; i++) {
        const Complaint *complaint = &complaints[i];
        if (fprintf(file, "%s\n%s\n%s\n%s\n%d\n%d\n",
                    complaint->id, complaint->category, complaint->problem,
                    complaint->location, (int)complaint->priority,
                    (int)complaint->status) < 0) {
            fprintf(stderr, "Could not write complaint data to %s.\n", DATA_FILE);
            fclose(file);
            return 0;
        }
    }

    if (fclose(file) != 0) {
        fprintf(stderr, "Could not finish saving %s.\n", DATA_FILE);
        return 0;
    }
    return 1;
}
```

```
static int findById(const char *id) {
    int i;
    for (i = 0; i < complaintCount; i++) {
        if (equalsIgnoreCase(complaints[i].id, id)) {
            return i;
        }
    }
    return -1;
}

static void printComplaint(const Complaint *complaint) {
    printf("%-14s %-18s %-10s %-14s %-24s %s\n",
           complaint->id, complaint->category, priorityName(complaint->priority),
           statusName(complaint->status), complaint->location, complaint->problem);
}

static void printTableHeader(void) {
    printf("%-14s %-18s %-10s %-14s %-24s %s\n",
           "ID", "Category", "Priority", "Status", "Location", "Problem");

printf("-----------------------------------------------------------------------------------------------------------\n");
}

static int registerComplaint(void) {
    Complaint complaint;
    char confirmation[8];
    int choice;
    int result;

    if (complaintCount >= MAX_COMPLAINTS) {
        printf("Complaint storage is full.\n");
        return 1;
    }

    if (readNonEmpty("Complaint ID: ", complaint.id, sizeof(complaint.id)) <= 0) {
        return 0;
    }
    if (findById(complaint.id) >= 0) {
        printf("A complaint with that ID already exists.\n");
        return 1;
    }
    if (readNonEmpty("Category (e.g., Laboratory, Hostel, Transport): ",
                     complaint.category, sizeof(complaint.category)) <= 0 ||
        readNonEmpty("Problem description: ", complaint.problem,
```

```
                sizeof(complaint.problem)) <= 0 ||
        readNonEmpty("Location: ", complaint.location,
                sizeof(complaint.location)) <= 0) {
            return 0;
        }

        printf("Severity / priority: 1. Critical  2. High  3. Medium  4. Low\n");
        if (!readInteger("Select priority: ", 1, 4, &choice)) {
            return 0;
        }
        complaint.priority = (Priority)choice;
        complaint.status = STATUS_PENDING;

        result = readLine("Register this complaint? (y/n): ", confirmation,
                 sizeof(confirmation));
        if (result <= 0) {
            return 0;
        }
        if (tolower((unsigned char)confirmation[0]) != 'y') {
            printf("Registration cancelled.\n");
            return 1;
        }

        complaints[complaintCount++] = complaint;
        if (saveComplaints()) {
            printf("Complaint registered with Pending status.\n");
        } else {
            printf("Complaint is in memory but could not be saved.\n");
        }
        return 1;
    }

    static void searchComplaint(void) {
        int choice;
        char query[LOCATION_SIZE];
        int i;
        int found = 0;

        printf("Search by: 1. ID  2. Category  3. Location\n");
        if (!readInteger("Select search type: ", 1, 3, &choice)) {
            return;
        }
        if (readNonEmpty("Search text: ", query, sizeof(query)) <= 0) {
            return;
```

```
    }

    if (choice == 1) {
        i = findById(query);
        if (i >= 0) {
            printTableHeader();
            printComplaint(&complaints[i]);
            found = 1;
        }
    } else {
        printTableHeader();
        for (i = 0; i < complaintCount; i++) {
            const char *field = choice == 2 ? complaints[i].category
                                            : complaints[i].location;
            if (containsIgnoreCase(field, query)) {
                printComplaint(&complaints[i]);
                found = 1;
            }
        }
    }
    if (!found) {
        printf("No matching complaints found.\n");
    }
}

static int comesBefore(const Complaint *left, const Complaint *right, int sortBy) {
    int comparison;
    if (sortBy == 1) {
        if (left->priority != right->priority) {
            return left->priority < right->priority;
        }
        return equalsIgnoreCase(left->id, right->id)
                ? 0
                : strcmp(left->id, right->id) < 0;
    }
    if (sortBy == 2) {
        comparison = tolower((unsigned char)left->id[0]) -
                     tolower((unsigned char)right->id[0]);
        if (comparison == 0) {
            comparison = strcmp(left->id, right->id);
        }
        return comparison < 0;
    }
    comparison = tolower((unsigned char)left->category[0]) -
```

```
            tolower((unsigned char)right->category[0]);
    if (comparison == 0) {
        comparison = strcmp(left->category, right->category);
    }
    return comparison < 0;
}

static void listComplaints(void) {
    Complaint sorted[MAX_COMPLAINTS];
    int sortBy;
    int i;

    printf("Sort by: 1. Priority (highest first)  2. ID  3. Category\n");
    if (!readInteger("Select sort order: ", 1, 3, &sortBy)) {
        return;
    }
    memcpy(sorted, complaints, (size_t)complaintCount * sizeof(Complaint));
    for (i = 1; i < complaintCount; i++) {
        Complaint current = sorted[i];
        int position = i;
        while (position > 0 &&
               comesBefore(&current, &sorted[position - 1], sortBy)) {
            sorted[position] = sorted[position - 1];
            position--;
        }
        sorted[position] = current;
    }

    if (complaintCount == 0) {
        printf("There are no registered complaints.\n");
        return;
    }
    printTableHeader();
    for (i = 0; i < complaintCount; i++) {
        printComplaint(&sorted[i]);
    }
}

static int updateStatus(void) {
    char id[ID_SIZE];
    int index;
    int status;

    if (readNonEmpty("Complaint ID to update: ", id, sizeof(id)) <= 0) {
```

```
        return 0;
    }
    index = findById(id);
    if (index < 0) {
        printf("Complaint not found.\n");
        return 1;
    }
    printf("Current status: %s\n", statusName(complaints[index].status));
    printf("New status: 1. Pending  2. In Progress  3. Resolved\n");
    if (!readInteger("Select new status: ", 1, 3, &status)) {
        return 0;
    }

    complaints[index].status = (Status)status;
    if (saveComplaints()) {
        printf("Complaint status updated to %s.\n",
               statusName(complaints[index].status));
    } else {
        printf("Status changed in memory but could not be saved.\n");
    }
    return 1;
}

static void showPending(void) {
    int i;
    int found = 0;

    for (i = 0; i < complaintCount; i++) {
        if (complaints[i].status == STATUS_PENDING) {
            if (!found) {
                printTableHeader();
            }
            printComplaint(&complaints[i]);
            found = 1;
        }
    }
    if (!found) {
        printf("There are no pending complaints.\n");
    }
}

static void filterComplaints(void) {
    int choice;
    int status = 0;
```

```
    char query[LOCATION_SIZE];
    int i;
    int found = 0;

    printf("Filter by: 1. Category  2. Location  3. Status\n");
    if (!readInteger("Select filter: ", 1, 3, &choice)) {
        return;
    }
    if (choice == 3) {
        printf("Status: 1. Pending  2. In Progress  3. Resolved\n");
        if (!readInteger("Select status: ", 1, 3, &status)) {
            return;
        }
    } else if (readNonEmpty(choice == 1 ? "Category contains: "
                                        : "Location contains: ",
                            query, sizeof(query)) <= 0) {
        return;
    }

    for (i = 0; i < complaintCount; i++) {
        int matches;
        if (choice == 1) {
            matches = containsIgnoreCase(complaints[i].category, query);
        } else if (choice == 2) {
            matches = containsIgnoreCase(complaints[i].location, query);
        } else {
            matches = complaints[i].status == (Status)status;
        }
        if (matches) {
            if (!found) {
                printTableHeader();
            }
            printComplaint(&complaints[i]);
            found = 1;
        }
    }
    if (!found) {
        printf("No matching complaints found.\n");
    }
}

static void showSummary(void) {
    char categories[MAX_COMPLAINTS][CATEGORY_SIZE];
    int categoryCounts[MAX_COMPLAINTS] = {0};
```

```
int categoryTotal = 0;
int priorityCounts[5] = {0};
int statusCounts[4] = {0};
int i;
int j;

for (i = 0; i < complaintCount; i++) {
    priorityCounts[complaints[i].priority]++;
    statusCounts[complaints[i].status]++;
    for (j = 0; j < categoryTotal; j++) {
        if (equalsIgnoreCase(categories[j], complaints[i].category)) {
            break;
        }
    }
    if (j == categoryTotal) {
        strcpy(categories[categoryTotal], complaints[i].category);
        categoryCounts[categoryTotal] = 0;
        categoryTotal++;
    }
    categoryCounts[j]++;
}

printf("\nComplaint Summary\n");
printf("-----------------\n");
printf("Total complaints: %d\n", complaintCount);
printf("Pending: %d | In Progress: %d | Resolved: %d\n",
       statusCounts[STATUS_PENDING], statusCounts[STATUS_IN_PROGRESS],
       statusCounts[STATUS_RESOLVED]);
if (complaintCount > 0) {
    printf("Resolution rate: %.1f%%\n",
           100.0 * statusCounts[STATUS_RESOLVED] / complaintCount);
} else {
    printf("Resolution rate: 0.0%%\n");
}
printf("Priority counts - Critical: %d, High: %d, Medium: %d, Low: %d\n",
       priorityCounts[PRIORITY_CRITICAL], priorityCounts[PRIORITY_HIGH],
       priorityCounts[PRIORITY_MEDIUM], priorityCounts[PRIORITY_LOW]);
printf("Department/category-wise counts:\n");
if (categoryTotal == 0) {
    printf("  No complaint data available.\n");
}
for (i = 0; i < categoryTotal; i++) {
    printf("  %-24s %d\n", categories[i], categoryCounts[i]);
}
```

```
}

static void printMenu(void) {
    printf("\n=== Smart Campus Complaint System ===\n");
    printf("1. Register a complaint\n");
    printf("2. Search complaints\n");
    printf("3. List complaints (sort)\n");
    printf("4. Update complaint status\n");
    printf("5. Display pending complaints\n");
    printf("6. Filter complaints\n");
    printf("7. Complaint summary / performance report\n");
    printf("0. Save and exit\n");
}

int main(void) {
    int choice;
    int keepRunning = 1;

    if (!loadComplaints()) {
        return EXIT_FAILURE;
    }
    printf("Loaded %d complaint(s).\n", complaintCount);

    while (keepRunning) {
        printMenu();
        if (!readInteger("Choose an option: ", 0, 7, &choice)) {
            printf("\nInput closed. Saving and exiting.\n");
            break;
        }
        switch (choice) {
            case 1:
                if (!registerComplaint()) {
                    keepRunning = 0;
                }
                break;
            case 2:
                searchComplaint();
                break;
            case 3:
                listComplaints();
                break;
            case 4:
                if (!updateStatus()) {
                    keepRunning = 0;
```

```
            }
            break;
        case 5:
            showPending();
            break;
        case 6:
            filterComplaints();
            break;
        case 7:
            showSummary();
            break;
        case 0:
            keepRunning = 0;
            break;
      }
   }

   if (!saveComplaints()) {
      return EXIT_FAILURE;
   }
   printf("Complaints saved. Goodbye.\n");
   return EXIT_SUCCESS;
}
```

output

```
Loaded 0 complaint(s).

=== Smart Campus Complaint System ===
1. Register a complaint
2. Search complaints
3. List complaints (sort)
4. Update complaint status
5. Display pending complaints
6. Filter complaints
7. Complaint summary / performance report
0. Save and exit
Choose an option: 1
Complaint ID: AA121
Category (e.g., Laboratory, Hostel, Transport): lab
Problem description: light not working
Location: cbe
Severity / priority: 1. Critical  2. High  3. Medium  4. Low
Select priority: 4
Register this complaint? (y/n): y
Complaint registered with Pending status.

=== Smart Campus Complaint System ===
1. Register a complaint
2. Search complaints
3. List complaints (sort)
4. Update complaint status
5. Display pending complaints
6. Filter complaints
7. Complaint summary / performance report
0. Save and exit
Choose an option: 1
Complaint ID: AA121
A complaint with that ID already exists.

=== Smart Campus Complaint System ===
1. Register a complaint
2. Search complaints
3. List complaints (sort)
4. Update complaint status
5. Display pending complaints
6. Filter complaints
7. Complaint summary / performance report
0. Save and exit
```

Scenario 3: smart hospital emergency queue system

```
#include <ctype.h>
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX_PATIENTS 500
#define MAX_DOCTORS 12
#define MAX_BEDS 30
#define ID_SIZE 32
#define NAME_SIZE 64
#define CONDITION_SIZE 160
#define DATA_FILE "hospital_patients.dat"

typedef enum {
    PRIORITY_CRITICAL = 1,
    PRIORITY_HIGH,
    PRIORITY_MEDIUM,
    PRIORITY_LOW
} Priority;

typedef enum {
    STATUS_WAITING = 1,
    STATUS_IN_TREATMENT,
    STATUS_ADMITTED,
    STATUS_DISCHARGED
} PatientStatus;

typedef struct {
    char id[ID_SIZE];
    char name[NAME_SIZE];
    int age;
    char condition[CONDITION_SIZE];
    Priority priority;
    PatientStatus status;
    int doctorIndex;
    int bedIndex;
    time_t registeredAt;
    time_t calledAt;
} Patient;
```

```
static Patient patients[MAX_PATIENTS];
static int patientCount;
static const char *doctorNames[MAX_DOCTORS] = {
    "Dr. Patel", "Dr. Rao", "Dr. Singh", "Dr. Kumar",
    "Dr. Shah", "Dr. Das", "Dr. Mehta", "Dr. Iyer",
    "Dr. Khan", "Dr. Roy", "Dr. Bose", "Dr. Nair"
};
static int doctorCount = 5;
static int bedCount = 10;

static int readLine(const char *prompt, char *buffer, size_t capacity) {
    size_t length;

    printf("%s", prompt);
    if (fgets(buffer, (int)capacity, stdin) == NULL) {
        return 0;
    }

    length = strcspn(buffer, "\r\n");
    if (buffer[length] == '\0' && length == capacity - 1) {
        int character;
        while ((character = getchar()) != '\n' && character != EOF) {
        }
        printf("Input is too long. Please try again.\n");
        return -1;
    }
    buffer[length] = '\0';
    return 1;
}

static int readRequired(const char *prompt, char *buffer, size_t capacity) {
    int result;
    do {
        result = readLine(prompt, buffer, capacity);
        if (result <= 0) {
            return result;
        }
        if (buffer[0] == '\0') {
            printf("This field cannot be empty.\n");
        }
    } while (buffer[0] == '\0');
    return 1;
}
```

```
static int readInteger(const char *prompt, int minimum, int maximum, int *value) {
    char buffer[64];
    char *end;
    long parsed;
    int result;

    for (;;) {
        result = readLine(prompt, buffer, sizeof(buffer));
        if (result == 0) {
            return 0;
        }
        if (result < 0) {
            continue;
        }
        errno = 0;
        parsed = strtol(buffer, &end, 10);
        while (isspace((unsigned char)*end)) {
            end++;
        }
        if (buffer[0] == '\0' || *end != '\0' || errno == ERANGE ||
            parsed < minimum || parsed > maximum) {
            printf("Enter a number from %d to %d.\n", minimum, maximum);
            continue;
        }
        *value = (int)parsed;
        return 1;
    }
}

static int equalsIgnoreCase(const char *left, const char *right) {
    while (*left != '\0' && *right != '\0') {
        if (tolower((unsigned char)*left) != tolower((unsigned char)*right)) {
            return 0;
        }
        left++;
        right++;
    }
    return *left == '\0' && *right == '\0';
}

static int containsIgnoreCase(const char *text, const char *query) {
    const char *start;
    size_t queryLength = strlen(query);
```

```
    for (start = text; *start != '\0'; start++) {
        size_t i = 0;
        while (i < queryLength && start[i] != '\0' &&
               tolower((unsigned char)start[i]) ==
                   tolower((unsigned char)query[i])) {
            i++;
        }
        if (i == queryLength) {
            return 1;
        }
    }
    return queryLength == 0;
}

static const char *priorityName(Priority priority) {
    switch (priority) {
        case PRIORITY_CRITICAL: return "Critical";
        case PRIORITY_HIGH: return "High";
        case PRIORITY_MEDIUM: return "Medium";
        case PRIORITY_LOW: return "Low";
        default: return "Unknown";
    }
}

static const char *statusName(PatientStatus status) {
    switch (status) {
        case STATUS_WAITING: return "Waiting";
        case STATUS_IN_TREATMENT: return "In Treatment";
        case STATUS_ADMITTED: return "Admitted";
        case STATUS_DISCHARGED: return "Discharged";
        default: return "Unknown";
    }
}

static int parseIntegerLine(FILE *file, int *value) {
    char line[64];
    char *end;
    long parsed;

    if (fgets(line, sizeof(line), file) == NULL) {
        return 0;
    }
    errno = 0;
```

```
    parsed = strtol(line, &end, 10);
    while (isspace((unsigned char)*end)) {
        end++;
    }
    if (line == end || *end != '\0' || errno == ERANGE ||
        parsed < -1 || parsed > 1000000) {
        return 0;
    }
    *value = (int)parsed;
    return 1;
}

static int parseTimeLine(FILE *file, time_t *value) {
    char line[64];
    char *end;
    long long parsed;

    if (fgets(line, sizeof(line), file) == NULL) {
        return 0;
    }
    errno = 0;
    parsed = strtoll(line, &end, 10);
    while (isspace((unsigned char)*end)) {
        end++;
    }
    if (line == end || *end != '\0' || errno == ERANGE || parsed < 0) {
        return 0;
    }
    *value = (time_t)parsed;
    return 1;
}

static int readStoredText(FILE *file, char *buffer, size_t capacity) {
    if (fgets(buffer, (int)capacity, file) == NULL) {
        return 0;
    }
    if (strchr(buffer, '\n') == NULL && !feof(file)) {
        int character;
        while ((character = fgetc(file)) != '\n' && character != EOF) {
        }
        return 0;
    }
    buffer[strcspn(buffer, "\r\n")] = '\0';
    return 1;
```

```
}

static int validPatient(const Patient *patient) {
    return patient->id[0] != '\0' && patient->name[0] != '\0' &&
           patient->condition[0] != '\0' && patient->age >= 0 &&
           patient->age <= 130 && patient->priority >= PRIORITY_CRITICAL &&
           patient->priority <= PRIORITY_LOW && patient->status >= STATUS_WAITING &&
           patient->status <= STATUS_DISCHARGED &&
           patient->doctorIndex >= -1 && patient->doctorIndex < doctorCount &&
           patient->bedIndex >= -1 && patient->bedIndex < bedCount;
}

static int loadPatients(void) {
    FILE *file = fopen(DATA_FILE, "r");
    char countLine[64];
    char *end;
    long parsedCount;
    int i;

    if (file == NULL) {
        if (errno == ENOENT) {
            return 1;
        }
        fprintf(stderr, "Cannot open %s: %s\n", DATA_FILE, strerror(errno));
        return 0;
    }

    if (fgets(countLine, sizeof(countLine), file) == NULL) {
        fprintf(stderr, "%s is empty or unreadable.\n", DATA_FILE);
        fclose(file);
        return 0;
    }
    errno = 0;
    parsedCount = strtol(countLine, &end, 10);
    while (isspace((unsigned char)*end)) {
        end++;
    }
    if (countLine == end || *end != '\0' || errno == ERANGE ||
        parsedCount < 0 || parsedCount > MAX_PATIENTS) {
        fprintf(stderr, "%s has an invalid patient count.\n", DATA_FILE);
        fclose(file);
        return 0;
    }
```

```
    for (i = 0; i < (int)parsedCount; i++) {
        Patient patient;
        int field;

        if (!readStoredText(file, patient.id, sizeof(patient.id)) ||
            !readStoredText(file, patient.name, sizeof(patient.name)) ||
            !parseIntegerLine(file, &patient.age) ||
            !readStoredText(file, patient.condition, sizeof(patient.condition)) ||
            !parseIntegerLine(file, &field)) {
            fprintf(stderr, "%s contains an incomplete patient record.\n", DATA_FILE);
            fclose(file);
            return 0;
        }
        patient.priority = (Priority)field;
        if (!parseIntegerLine(file, &field)) {
            fprintf(stderr, "%s contains an incomplete patient record.\n", DATA_FILE);
            fclose(file);
            return 0;
        }
        patient.status = (PatientStatus)field;
        if (!parseIntegerLine(file, &patient.doctorIndex) ||
            !parseIntegerLine(file, &patient.bedIndex) ||
            !parseTimeLine(file, &patient.registeredAt) ||
            !parseTimeLine(file, &patient.calledAt) ||
            !validPatient(&patient)) {
            fprintf(stderr, "%s contains an invalid patient record.\n", DATA_FILE);
            fclose(file);
            return 0;
        }
        patients[i] = patient;
    }

    if (fclose(file) != 0) {
        fprintf(stderr, "Could not close %s after reading.\n", DATA_FILE);
        return 0;
    }
    patientCount = (int)parsedCount;
    return 1;
}

static int savePatients(void) {
    FILE *file = fopen(DATA_FILE, "w");
    int i;
```

```
    if (file == NULL) {
        fprintf(stderr, "Cannot save %s: %s\n", DATA_FILE, strerror(errno));
        return 0;
    }
    if (fprintf(file, "%d\n", patientCount) < 0) {
        fprintf(stderr, "Could not write patient data to %s.\n", DATA_FILE);
        fclose(file);
        return 0;
    }
    for (i = 0; i < patientCount; i++) {
        const Patient *patient = &patients[i];
        if (fprintf(file, "%s\n%s\n%d\n%s\n%d\n%d\n%d\n%d\n%lld\n%lld\n",
                    patient->id, patient->name, patient->age, patient->condition,
                    (int)patient->priority, (int)patient->status,
                    patient->doctorIndex, patient->bedIndex,
                    (long long)patient->registeredAt,
                    (long long)patient->calledAt) < 0) {
            fprintf(stderr, "Could not write patient data to %s.\n", DATA_FILE);
            fclose(file);
            return 0;
        }
    }
    if (fclose(file) != 0) {
        fprintf(stderr, "Could not finish saving %s.\n", DATA_FILE);
        return 0;
    }
    return 1;
}

static int findPatient(const char *id) {
    int i;
    for (i = 0; i < patientCount; i++) {
        if (equalsIgnoreCase(patients[i].id, id)) {
            return i;
        }
    }
    return -1;
}

static int doctorBusy(int doctorIndex) {
    int i;
    for (i = 0; i < patientCount; i++) {
        if (patients[i].doctorIndex == doctorIndex &&
            patients[i].status == STATUS_IN_TREATMENT) {
```

```
            return 1;
        }
    }
    return 0;
}

static int bedOccupied(int bedIndex) {
    int i;
    for (i = 0; i < patientCount; i++) {
        if (patients[i].bedIndex == bedIndex &&
            patients[i].status != STATUS_DISCHARGED) {
            return 1;
        }
    }
    return 0;
}

static int availableDoctor(void) {
    int i;
    for (i = 0; i < doctorCount; i++) {
        if (!doctorBusy(i)) {
            return i;
        }
    }
    return -1;
}

static int availableBed(void) {
    int i;
    for (i = 0; i < bedCount; i++) {
        if (!bedOccupied(i)) {
            return i;
        }
    }
    return -1;
}

static const char *doctorNameFor(const Patient *patient) {
    return patient->doctorIndex < 0 ? "None" : doctorNames[patient->doctorIndex];
}

static void printPatient(const Patient *patient) {
    char bed[12];
    if (patient->bedIndex < 0) {
```

```
        strcpy(bed, "None");
    } else {
        snprintf(bed, sizeof(bed), "B%02d", patient->bedIndex + 1);
    }
    printf("%-12s %-18s %3d %-12s %-14s %-15s %-13s %s\n",
           patient->id, patient->name, patient->age, priorityName(patient->priority),
           statusName(patient->status), doctorNameFor(patient), bed,
           patient->condition);
}

static void printPatientHeader(void) {
    printf("%-12s %-18s %3s %-12s %-14s %-15s %-13s %s\n",
           "Patient ID", "Name", "Age", "Priority", "Status", "Doctor", "Bed",
           "Condition");

printf("-------------------------------------------------------------------------------------------------------------------\n");
}

static int registerPatient(void) {
    Patient patient;
    int choice;
    int result;

    if (patientCount >= MAX_PATIENTS) {
        printf("Patient storage is full.\n");
        return 1;
    }
    if (readRequired("Patient ID: ", patient.id, sizeof(patient.id)) <= 0) {
        return 0;
    }
    if (findPatient(patient.id) >= 0) {
        printf("A patient with that ID already exists.\n");
        return 1;
    }
    if (readRequired("Patient name: ", patient.name, sizeof(patient.name)) <= 0 ||
        !readInteger("Age: ", 0, 130, &patient.age) ||
        readRequired("Condition / emergency description: ", patient.condition,
                     sizeof(patient.condition)) <= 0) {
        return 0;
    }
    printf("Priority: 1. Critical  2. High  3. Medium  4. Low\n");
    if (!readInteger("Assign priority: ", 1, 4, &choice)) {
        return 0;
    }
```

```
    patient.priority = (Priority)choice;
    patient.status = STATUS_WAITING;
    patient.doctorIndex = -1;
    patient.bedIndex = -1;
    patient.registeredAt = time(NULL);
    patient.calledAt = 0;
    if (patient.registeredAt == (time_t)-1) {
        fprintf(stderr, "Could not read the system clock; patient not registered.\n");
        return 1;
    }

    patients[patientCount++] = patient;
    result = savePatients();
    if (result) {
        printf("Patient registered and placed in the Waiting queue.\n");
    } else {
        printf("Patient is in memory but could not be saved.\n");
    }
    return 1;
}

static int isBeforeInQueue(const Patient *left, const Patient *right) {
    if (left->priority != right->priority) {
        return left->priority < right->priority;
    }
    if (left->registeredAt != right->registeredAt) {
        return left->registeredAt < right->registeredAt;
    }
    return 0;
}

static int callNextPatient(void) {
    int best = -1;
    int doctor;
    int i;

    for (i = 0; i < patientCount; i++) {
        if (patients[i].status == STATUS_WAITING &&
            (best < 0 || isBeforeInQueue(&patients[i], &patients[best]))) {
            best = i;
        }
    }
    if (best < 0) {
```

```
        printf("There are no patients waiting.\n");
        return 1;
    }
    doctor = availableDoctor();
    if (doctor < 0) {
        printf("No doctor is currently available. The patient remains in the queue.\n");
        return 1;
    }

    patients[best].doctorIndex = doctor;
    patients[best].status = STATUS_IN_TREATMENT;
    patients[best].calledAt = time(NULL);
    if (patients[best].calledAt == (time_t)-1) {
        patients[best].calledAt = patients[best].registeredAt;
    }
    printf("Calling %s (%s, %s); assigned to %s.\n",
           patients[best].name, patients[best].id,
           priorityName(patients[best].priority), doctorNames[doctor]);
    if (!savePatients()) {
        printf("Patient was called in memory but the change could not be saved.\n");
    }
    return 1;
}

static int updatePatientStatus(void) {
    char id[ID_SIZE];
    int index;
    int choice;

    if (readRequired("Patient ID: ", id, sizeof(id)) <= 0) {
        return 0;
    }
    index = findPatient(id);
    if (index < 0) {
        printf("Patient not found.\n");
        return 1;
    }
    printf("Current status: %s\n", statusName(patients[index].status));
    printf("New status: 1. Waiting  2. In Treatment  3. Admitted  4. Discharged\n");
    if (!readInteger("Select status: ", 1, 4, &choice)) {
        return 0;
    }

    if (choice == STATUS_IN_TREATMENT &&
```

```
        patients[index].status != STATUS_IN_TREATMENT) {
        int doctor = availableDoctor();
        if (doctor < 0) {
            printf("No doctor is available; status was not changed.\n");
            return 1;
        }
        patients[index].doctorIndex = doctor;
        patients[index].calledAt = time(NULL);
        if (patients[index].calledAt == (time_t)-1) {
            patients[index].calledAt = patients[index].registeredAt;
        }
    }
    patients[index].status = (PatientStatus)choice;
    if (choice == STATUS_WAITING) {
        patients[index].doctorIndex = -1;
        patients[index].bedIndex = -1;
        patients[index].calledAt = 0;
    } else if (choice == STATUS_ADMITTED || choice == STATUS_DISCHARGED) {
        patients[index].doctorIndex = -1;
        if (choice == STATUS_DISCHARGED) {
            patients[index].bedIndex = -1;
        }
    }

    if (savePatients()) {
        printf("Patient status updated to %s.\n", statusName(patients[index].status));
    } else {
        printf("Status changed in memory but could not be saved.\n");
    }
    return 1;
}

static int assignResource(void) {
    char id[ID_SIZE];
    int index;
    int resourceType;
    int resourceChoice;

    if (readRequired("Patient ID: ", id, sizeof(id)) <= 0) {
        return 0;
    }
    index = findPatient(id);
    if (index < 0) {
        printf("Patient not found.\n");
```

```
    return 1;
}
if (patients[index].status == STATUS_DISCHARGED) {
    printf("Resources cannot be assigned to a discharged patient.\n");
    return 1;
}
printf("Assign: 1. Doctor  2. Bed\n");
if (!readInteger("Select resource: ", 1, 2, &resourceType)) {
    return 0;
}
if (resourceType == 1) {
    int freeDoctor = availableDoctor();
    if (patients[index].doctorIndex >= 0 &&
        patients[index].status == STATUS_IN_TREATMENT) {
        printf("Patient already has %s assigned.\n", doctorNameFor(&patients[index]));
        return 1;
    }
    if (freeDoctor < 0) {
        printf("No doctor is currently available.\n");
        return 1;
    }
    patients[index].doctorIndex = freeDoctor;
    if (patients[index].status == STATUS_WAITING) {
        patients[index].status = STATUS_IN_TREATMENT;
        patients[index].calledAt = time(NULL);
        if (patients[index].calledAt == (time_t)-1) {
            patients[index].calledAt = patients[index].registeredAt;
        }
    }
    printf("Assigned %s to %s.\n", doctorNames[freeDoctor], patients[index].name);
} else {
    int freeBed = availableBed();
    if (patients[index].bedIndex >= 0) {
        printf("Patient already has bed B%02d assigned.\n",
               patients[index].bedIndex + 1);
        return 1;
    }
    if (freeBed < 0) {
        printf("No bed is currently available.\n");
        return 1;
    }
    if (!readInteger("Bed number to assign (0 for first available): ",
                     0, bedCount, &resourceChoice)) {
        return 0;
```

```
        }
        if (resourceChoice == 0) {
            resourceChoice = freeBed + 1;
        }
        if (bedOccupied(resourceChoice - 1)) {
            printf("That bed is occupied; no assignment was made.\n");
            return 1;
        }
        patients[index].bedIndex = resourceChoice - 1;
        printf("Assigned bed B%02d to %s.\n", resourceChoice, patients[index].name);
    }
    if (!savePatients()) {
        printf("Resource assignment is in memory but could not be saved.\n");
    }
    return 1;
}

static int sortBefore(const Patient *left, const Patient *right, int criterion) {
    if (criterion == 1) {
        return isBeforeInQueue(left, right);
    }
    if (criterion == 2) {
        return strcmp(left->id, right->id) < 0;
    }
    if (left->age != right->age) {
        return left->age > right->age;
    }
    return strcmp(left->id, right->id) < 0;
}

static void listPatients(void) {
    Patient sorted[MAX_PATIENTS];
    int criterion;
    int i;

    printf("Sort by: 1. Priority (critical first)  2. Patient ID  3. Age (oldest first)\n");
    if (!readInteger("Select sort order: ", 1, 3, &criterion)) {
        return;
    }
    if (patientCount == 0) {
        printf("There are no registered patients.\n");
        return;
    }
    memcpy(sorted, patients, (size_t)patientCount * sizeof(Patient));
```

```
    for (i = 1; i < patientCount; i++) {
        Patient current = sorted[i];
        int position = i;
        while (position > 0 && sortBefore(&current, &sorted[position - 1], criterion)) {
            sorted[position] = sorted[position - 1];
            position--;
        }
        sorted[position] = current;
    }
    printPatientHeader();
    for (i = 0; i < patientCount; i++) {
        printPatient(&sorted[i]);
    }
}

static void showWaitingQueue(void) {
    Patient waiting[MAX_PATIENTS];
    int count = 0;
    int i;
    int j;

    for (i = 0; i < patientCount; i++) {
        if (patients[i].status == STATUS_WAITING) {
            waiting[count++] = patients[i];
        }
    }
    if (count == 0) {
        printf("There are no patients waiting.\n");
        return;
    }
    for (i = 1; i < count; i++) {
        Patient current = waiting[i];
        j = i;
        while (j > 0 && isBeforeInQueue(&current, &waiting[j - 1])) {
            waiting[j] = waiting[j - 1];
            j--;
        }
        waiting[j] = current;
    }
    printPatientHeader();
    for (i = 0; i < count; i++) {
        printPatient(&waiting[i]);
    }
}
```

```
static void searchPatient(void) {
    char query[CONDITION_SIZE];
    int criterion;
    int i;
    int found = 0;

    printf("Search by: 1. Patient ID  2. Name  3. Condition\n");
    if (!readInteger("Select search type: ", 1, 3, &criterion) ||
        readRequired("Search text: ", query, sizeof(query)) <= 0) {
        return;
    }
    if (criterion == 1) {
        i = findPatient(query);
        if (i >= 0) {
            printPatientHeader();
            printPatient(&patients[i]);
            found = 1;
        }
    } else {
        printPatientHeader();
        for (i = 0; i < patientCount; i++) {
            const char *field = criterion == 2 ? patients[i].name : patients[i].condition;
            if (containsIgnoreCase(field, query)) {
                printPatient(&patients[i]);
                found = 1;
            }
        }
    }
    if (!found) {
        printf("No matching patients found.\n");
    }
}

static void showResources(void) {
    int i;
    printf("\nDoctors\n");
    for (i = 0; i < doctorCount; i++) {
        int j;
        const char *assignedTo = NULL;
        for (j = 0; j < patientCount; j++) {
            if (patients[j].doctorIndex == i &&
                patients[j].status == STATUS_IN_TREATMENT) {
                assignedTo = patients[j].name;
```

```
                break;
            }
        }
        printf("  %-12s %s", doctorNames[i], assignedTo == NULL ? "Available" : "Busy");
        if (assignedTo != NULL) {
            printf(" (patient: %s)", assignedTo);
        }
        putchar('\n');
    }
    printf("Beds\n");
    for (i = 0; i < bedCount; i++) {
        int j;
        const char *assignedTo = NULL;
        for (j = 0; j < patientCount; j++) {
            if (patients[j].bedIndex == i && patients[j].status != STATUS_DISCHARGED) {
                assignedTo = patients[j].name;
                break;
            }
        }
        printf("  B%02d: %s", i + 1, assignedTo == NULL ? "Available" : "Occupied");
        if (assignedTo != NULL) {
            printf(" (patient: %s)", assignedTo);
        }
        putchar('\n');
    }
}

static void showSummary(void) {
    int priorityCounts[5] = {0};
    int statusCounts[5] = {0};
    double totalWaitSeconds = 0.0;
    int treatedCount = 0;
    time_t now = time(NULL);
    int i;

    for (i = 0; i < patientCount; i++) {
        const Patient *patient = &patients[i];
        priorityCounts[patient->priority]++;
        statusCounts[patient->status]++;
        if (patient->calledAt > 0 && patient->calledAt >= patient->registeredAt) {
            totalWaitSeconds += difftime(patient->calledAt, patient->registeredAt);
            treatedCount++;
        } else if (patient->status == STATUS_WAITING && now >= patient->registeredAt) {
            totalWaitSeconds += difftime(now, patient->registeredAt);
```

```
            treatedCount++;
        }
    }

    printf("\nEmergency Department Summary\n");
    printf("---------------------------\n");
    printf("Total patients: %d\n", patientCount);
    printf("Waiting: %d | In Treatment: %d | Admitted: %d | Discharged: %d\n",
           statusCounts[STATUS_WAITING], statusCounts[STATUS_IN_TREATMENT],
           statusCounts[STATUS_ADMITTED], statusCounts[STATUS_DISCHARGED]);
    printf("Priority counts - Critical: %d, High: %d, Medium: %d, Low: %d\n",
           priorityCounts[PRIORITY_CRITICAL], priorityCounts[PRIORITY_HIGH],
           priorityCounts[PRIORITY_MEDIUM], priorityCounts[PRIORITY_LOW]);
    if (treatedCount > 0) {
        printf("Average wait to call / current wait: %.1f minutes\n",
               totalWaitSeconds / treatedCount / 60.0);
    } else {
        printf("Average wait to call / current wait: 0.0 minutes\n");
    }
    {
        int freeDoctors = 0;
        int freeBeds = 0;
        for (i = 0; i < doctorCount; i++) {
            if (!doctorBusy(i)) {
                freeDoctors++;
            }
        }
        for (i = 0; i < bedCount; i++) {
            if (!bedOccupied(i)) {
                freeBeds++;
            }
        }
        printf("Currently available: %d doctors, %d beds\n", freeDoctors, freeBeds);
    }
}

static void printMenu(void) {
    printf("\n=== Smart Hospital Emergency Queue ===\n");
    printf("1. Register a patient\n");
    printf("2. Display patients (sort)\n");
    printf("3. Call next highest-priority patient\n");
    printf("4. Assign an available doctor or bed\n");
    printf("5. Update patient status\n");
    printf("6. Search patients\n");
```

```
    printf("7. Display waiting queue\n");
    printf("8. Show doctor and bed availability\n");
    printf("9. Emergency department summary\n");
    printf("0. Save and exit\n");
}

int main(void) {
    int choice;
    int running = 1;

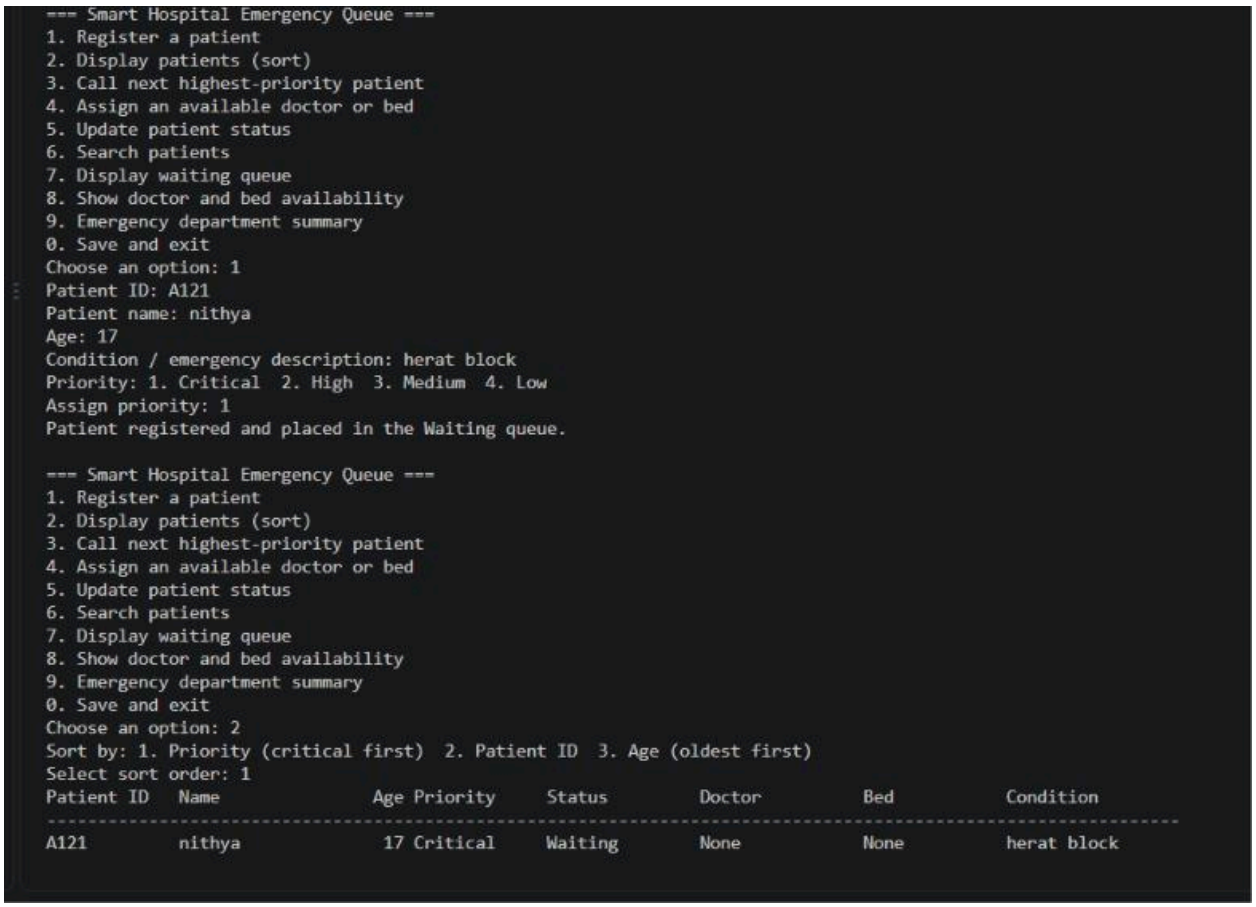
    if (!loadPatients()) {
        return EXIT_FAILURE;
    }
    printf("Loaded %d patient record(s). Doctors: %d; beds: %d.\n",
           patientCount, doctorCount, bedCount);

    while (running) {
        printMenu();
        if (!readInteger("Choose an option: ", 0, 9, &choice)) {
            printf("\nInput closed; saving and exiting.\n");
            break;
        }
        switch (choice) {
            case 1:
                if (!registerPatient()) running = 0;
                break;
            case 2:
                listPatients();
                break;
            case 3:
                callNextPatient();
                break;
            case 4:
                if (!assignResource()) running = 0;
                break;
            case 5:
                if (!updatePatientStatus()) running = 0;
                break;
            case 6:
                searchPatient();
                break;
            case 7:
                showWaitingQueue();
                break;
```

```
            case 8:
                showResources();
                break;
            case 9:
                showSummary();
                break;
            case 0:
                running = 0;
                break;
        }
    }

    if (!savePatients()) {
        return EXIT_FAILURE;
    }
    printf("Patient records saved. Goodbye.\n");
    return EXIT_SUCCESS;
}
```

Output



```
=== Smart Hospital Emergency Queue ===
1. Register a patient
2. Display patients (sort)
3. Call next highest-priority patient
4. Assign an available doctor or bed
5. Update patient status
6. Search patients
7. Display waiting queue
8. Show doctor and bed availability
9. Emergency department summary
0. Save and exit
Choose an option: 1
Patient ID: A121
Patient name: nithya
Age: 17
Condition / emergency description: herat block
Priority: 1. Critical  2. High  3. Medium  4. Low
Assign priority: 1
Patient registered and placed in the Waiting queue.

=== Smart Hospital Emergency Queue ===
1. Register a patient
2. Display patients (sort)
3. Call next highest-priority patient
4. Assign an available doctor or bed
5. Update patient status
6. Search patients
7. Display waiting queue
8. Show doctor and bed availability
9. Emergency department summary
0. Save and exit
Choose an option: 2
Sort by: 1. Priority (critical first)  2. Patient ID  3. Age (oldest first)
Select sort order: 1
Patient ID   Name              Age Priority     Status         Doctor          Bed           Condition
-------------------------------------------------------------------------------------------------------
A121         nithya             17 Critical     Waiting        None            None          herat block
```

```
=== Smart Hospital Emergency Queue ===
1. Register a patient
2. Display patients (sort)
3. Call next highest-priority patient
4. Assign an available doctor or bed
5. Update patient status
6. Search patients
7. Display waiting queue
8. Show doctor and bed availability
9. Emergency department summary
0. Save and exit
Choose an option: 3
Calling nithya (A121, Critical); assigned to Dr. Patel.

=== Smart Hospital Emergency Queue ===
1. Register a patient
2. Display patients (sort)
3. Call next highest-priority patient
4. Assign an available doctor or bed
5. Update patient status
6. Search patients
7. Display waiting queue
8. Show doctor and bed availability
9. Emergency department summary
0. Save and exit
Choose an option: 4
Patient ID: AA121
Patient not found.

=== Smart Hospital Emergency Queue ===
1. Register a patient
2. Display patients (sort)
3. Call next highest-priority patient
4. Assign an available doctor or bed
5. Update patient status
6. Search patients
7. Display waiting queue
8. Show doctor and bed availability
```

```
Select search type: 1
Search text: AA121
No matching patients found.

=== Smart Hospital Emergency Queue ===
1. Register a patient
2. Display patients (sort)
3. Call next highest-priority patient
4. Assign an available doctor or bed
5. Update patient status
6. Search patients
7. Display waiting queue
8. Show doctor and bed availability
9. Emergency department summary
0. Save and exit
Choose an option: 0
Patient records saved. Goodbye.
PS C:\Users\NETRA MS\OneDrive\Desktop\c codecraft 26>
```